

```

"""
AK SIGNAL BOT - engine
Indicators: EMA 9/21/50, RSI, MACD, Stochastic,
Bollinger Bands, ATR,
Support/Resistance (pivot clusters), candle
patterns.
Only needs pandas + numpy.
"""

import numpy as np
import pandas as pd

# -----
# ----- indicators

def ema(s, n):
    return s.ewm(span=n, adjust=False).mean()

def rsi(close, n=14):
    d = close.diff()
    up = d.clip(lower=0).ewm(alpha=1 / n,
adjust=False).mean()
    dn = (-d.clip(upper=0)).ewm(alpha=1 / n,
adjust=False).mean()
    rs = up / dn.replace(0, np.nan)
    return (100 - 100 / (1 + rs)).fillna(50)

def atr(df, n=14):
    pc = df["Close"].shift()
    tr = pd.concat([df["High"] - df["Low"],
(df["High"] - pc).abs(),
(df["Low"] - pc).abs()],
axis=1).max(axis=1)

```

```

        return tr.ewm(alpha=1 / n,
adjust=False).mean()

def add_indicators(df, p=None):
    p = p or {}
    d = df.copy()
    c = d["Close"]
    d["ema9"] = ema(c, p.get("ema_fast", 9))
    d["ema21"] = ema(c, p.get("ema_mid", 21))
    d["ema50"] = ema(c, p.get("ema_slow", 50))
    d["rsi"] = rsi(c, p.get("rsi_len", 14))
    m = ema(c, 12) - ema(c, 26)
    d["macd"], d["macd_sig"] = m, ema(m, 9)
    d["macd_hist"] = d["macd"] - d["macd_sig"]
    lo, hi = d["Low"].rolling(14).min(),
d["High"].rolling(14).max()
    d["stoch_k"] = ((c - lo) / (hi -
lo).replace(0, np.nan) * 100).fillna(50)
    d["stoch_d"] =
d["stoch_k"].rolling(3).mean()
    mid = c.rolling(20).mean()
    sd = c.rolling(20).std()
    d["bb_mid"], d["bb_up"], d["bb_low"] = mid,
mid + 2 * sd, mid - 2 * sd
    d["atr"] = atr(d)
    w = p.get("pivot_w", 3)
    d["piv_h"] = d["High"] ==
d["High"].rolling(2 * w + 1, center=True).max()
    d["piv_l"] = d["Low"] == d["Low"].rolling(2
* w + 1, center=True).min()
    return d

# -----
----- support/resistance
def sr_levels(d, t, lookback=150, w=3):

```

```

        """Levels known at bar t (only pivots
        already confirmed, no look-ahead)."""
        lo_i = max(0, t - lookback)
        hi_i = t - w
        if hi_i <= lo_i:
            return []
        seg = d.iloc[lo_i:hi_i + 1]
        pts = list(seg.loc[seg["piv_h"], "High"]) +
list(seg.loc[seg["piv_l"], "Low"])
        if not pts:
            return []
        tol = max(float(d["atr"].iloc[t]) * 0.6, 1e-
12)
        pts.sort()
        groups, cur = [], [pts[0]]
        for x in pts[1:]:
            if x - np.mean(cur) <= tol:
                cur.append(x)
            else:
                groups.append(cur)
                cur = [x]
        groups.append(cur)
        return [(float(np.mean(g)), len(g)) for g in
groups] # (price, touches)

# -----
----- signals
MAX_SCORE = 12

def score_at(d, t, p=None):
    p = p or {}
    r, q = d.iloc[t], d.iloc[t - 1]
    price, a = float(r["Close"]),
float(r["atr"])
    votes = []

```

```

def vote(name, v, note):
    votes.append({"name": name, "vote": v,
"note": note})

# EMA trend (2)
if r.ema9 > r.ema21 > r.ema50:
    vote("EMA trend", 2, "9 > 21 > 50,
uptrend")
elif r.ema9 < r.ema21 < r.ema50:
    vote("EMA trend", -2, "9 < 21 < 50,
downtrend")
elif r.ema9 > r.ema21:
    vote("EMA trend", 1, "Fast EMA above mid
EMA")
elif r.ema9 < r.ema21:
    vote("EMA trend", -1, "Fast EMA below
mid EMA")
else:
    vote("EMA trend", 0, "Flat")

# RSI (2)
lo_, hi_ = p.get("rsi_low", 30),
p.get("rsi_high", 70)
if r.rsi < lo_:
    vote("RSI", 2, f"{r.rsi:.0f}, oversold")
elif r.rsi > hi_:
    vote("RSI", -2, f"{r.rsi:.0f},
overbought")
elif r.rsi < 40 and r.rsi > q.rsi:
    vote("RSI", 1, f"{r.rsi:.0f}, rising
from low zone")
elif r.rsi > 60 and r.rsi < q.rsi:
    vote("RSI", -1, f"{r.rsi:.0f}, falling
from high zone")
else:
    vote("RSI", 0, f"{r.rsi:.0f}, neutral")

```

```

# MACD (1)
if r.macd_hist > 0 and r.macd_hist >
q.macd_hist:
    vote("MACD", 1, "Histogram positive and
growing")
    elif r.macd_hist < 0 and r.macd_hist <
q.macd_hist:
        vote("MACD", -1, "Histogram negative and
falling")
    else:
        vote("MACD", 0, "No clear momentum")

# Stochastic (1)
if r.stoch_k < 20 and r.stoch_k > r.stoch_d:
    vote("Stochastic", 1, "Oversold, K
crossing up")
    elif r.stoch_k > 80 and r.stoch_k <
r.stoch_d:
        vote("Stochastic", -1, "Overbought, K
crossing down")
    else:
        vote("Stochastic", 0, f"K
{r.stoch_k:.0f}")

# Bollinger (1)
if price < r.bb_low:
    vote("Bollinger", 1, "Price below lower
band")
    elif price > r.bb_up:
        vote("Bollinger", -1, "Price above upper
band")
    else:
        vote("Bollinger", 0, "Inside bands")

# Support / Resistance (3)
levels = sr_levels(d, t,

```

```

p.get("sr_lookback", 150), p.get("pivot_w", 3))
    sup = [x for x in levels if x[0] <= price]
    res = [x for x in levels if x[0] > price]
    ns = max(sup, key=lambda x: x[0]) if sup
else None
    nr = min(res, key=lambda x: x[0]) if res
else None
    near = a * 0.7
    sr_v, sr_n = 0, "No level nearby"
    if ns and price - ns[0] <= near:
        sr_v, sr_n = (3 if ns[1] >= 2 else 2),
f"At support {ns[0]:.5g} ({ns[1]} touches)"
    if nr and nr[0] - price <= near:
        if abs(sr_v) == 0 or (nr[0] - price) <
(price - ns[0] if ns else 1e9):
            sr_v, sr_n = (-3 if nr[1] >= 2 else
-2), f"At resistance {nr[0]:.5g} ({nr[1]}
touches)"
    vote("Support/Resistance", sr_v, sr_n)

# Candle pattern (2)
body = abs(r.Close - r.Open)
rng = max(r.High - r.Low, 1e-12)
lw = min(r.Open, r.Close) - r.Low
uw = r.High - max(r.Open, r.Close)
bull_eng = r.Close > r.Open and q.Close <
q.Open and r.Close >= q.Open and r.Open <=
q.Close
bear_eng = r.Close < r.Open and q.Close >
q.Open and r.Close <= q.Open and r.Open >=
q.Close
if bull_eng:
    vote("Candle", 2, "Bullish engulfing")
elif bear_eng:
    vote("Candle", -2, "Bearish engulfing")
elif lw > 2 * body and lw > 0.55 * rng:
    vote("Candle", 1, "Hammer / long lower

```

```

wick")
    elif uw > 2 * body and uw > 0.55 * rng:
        vote("Candle", -1, "Shooting star / long
upper wick")
    else:
        vote("Candle", 0, "No pattern")

    total = sum(v["vote"] for v in votes)
    return {
        "score": total, "votes": votes, "price":
price, "atr": a,
        "support": ns, "resistance": nr,
"levels": levels,
    }

def decide(score, threshold=5):
    if score >= threshold:
        return "CALL"
    if score <= -threshold:
        return "PUT"
    return "WAIT"

def analyze(df, p=None, threshold=5):
    """Signal for the last fully closed
candle."""
    d = add_indicators(df, p)
    t = len(d) - 1
    res = score_at(d, t, p)
    res["signal"] = decide(res["score"],
threshold)
    res["strength"] = int(min(100,
abs(res["score"]) / MAX_SCORE * 100))
    res["time"] = d.index[t]
    res["df"] = d
    return res

```

```

# -----
----- backtest
def backtest(df, expiry=1, threshold=5,
payout=0.85, p=None, maxBars=1500):
    d = add_indicators(df,
p).tail(maxBars).reset_index(drop=False)
    d.index = d.index # positional
    wins = losses = 0
    rows = []
    for t in range(60, len(d) - expiry):
        s = score_at(d, t, p)["score"]
        sig = decide(s, threshold)
        if sig == "WAIT":
            continue
        entry, exit_ = d["Close"].iloc[t],
d["Close"].iloc[t + expiry]
        win = (exit_ > entry) if sig == "CALL"
else (exit_ < entry)
        wins += int(win)
        losses += int(not win)
    n = wins + losses
    wr = wins / n * 100 if n else 0.0
    be = 100 / (1 + payout)
    profit_units = wins * payout - losses # in
units of stake
    return {"trades": n, "wins": wins, "losses":
losses, "win_rate": wr,
        "break_even": be, "net_units":
profit_units}

```